



COMPARISON OF Q-LEARNING AND DEEP Q-NETWORK FOR AUTONOMOUS NAVIGATION IN GRID-BASED ENVIRONMENTS

Abstract:

Obstacle-avoidance navigation is one of the primary problems of mobile robotics and smart transportation systems. This paper involves a comparison of the performance of classical Q-learning and Deep Q-Network (DQN) algorithms in learning navigation policies in a 10x10 grid world with fixed obstacles. The agent should begin at a stationary initial point and move to an objective point avoiding collisions as well as the shortest possible path. Q-learning attained a performance success of 99.11% on an average path length of 28.51 ± 19.64 steps, which indicates an expedited convergence and a dependable operation. Conversely, DQN achieved a success rate of 52.52 percent and took 293.6 episodes to converge and exhibit more variation and slow down learning. The findings suggest that tabular Q-learning is more efficient than deep reinforcement learning in small and discrete state spaces whereas DQN can be used with the benefit of larger or continuous areas where the use of function approximation is needed.

Keywords:

Q-Learning, Deep Q-Network (DQN), Autonomous Navigation, Path Planning, Deep Reinforcement Learning, Obstacle Avoidance..

Information about the authors

Dr. Zahraa Hussein Ameen Aldahlaki

Computer Engineering Technology, AL Nukhba University College
zahraaalahlaki92@gmail.com

Hussain Fadhil Hadi kaze

Computer Engineering Technology, AL Nukhba University College
free73293@gmail.com

Sajjad Sattar Jabbar

Computer Engineering Technology, AL Nukhba University College
Sajaadzh@gmail.com

Rsool Salem Hassan

Computer Engineering Technology, AL Nukhba University College
rswlsalm61@gmail.com



Introduction

The independent navigation under cluttered conditions is the basic feature of mobile robots and smart cars. The agent in a grid based environment has to navigate to a target position by avoiding obstacles and unnecessary travelling time, has to confront uncertain conditions within its environment, and has to navigate by following a grid. Classical path-planning algorithms, like the A* and Dijkstra algorithm, are able to calculate optimal paths under conditions of complete information of the environment, but are not inherently adaptive to learning during interaction and to changing the environment. The alternative of reinforcing learning (RL) is a powerful process as it enables an agent to acquire an optimal policy directly through trial-and-error.

Tabular Q-learning is one of the most popular algorithms of discrete state-action space among the RL techniques. It dynamically optimizes a Q-table which contains the expected payoff of each action in each state, and has been applied to several navigation and control problems successfully [1]. Tabular methods are however not practical when the state space is large or continuous due to the rapid increase in the number of table entries. Deep reinforcement learning overcomes this limitation by estimating the Q-function by a neural network, allowing the generalization of similar states and more complex observation spaces [2]. One of the most notable examples that unify Q-learning with deep neural networks, experience replay, and target networks is the Deep Q-Network (DQN) algorithm [3].

This paper examines and compares the performance of Q-learning and DQN on autonomous navigation in a fixed 10 x 10 gridworld and obstacles. The agent is given a fixed initial position and has to arrive at a predetermined goal without collisions, and is rewarded highly on success and severely punished on failure and excessive steps. This research primarily aims to assess the ability of each algorithm to learn an effective navigation policy in a small yet non-trivial environment, as well as to examine their convergence behavior, success, and path efficiency. The findings can inform future research on the practical trade-offs of using classical tabular RL versus deep RL in discrete navigation problems and inspire future work in larger or more realistic autonomous driving environments.

Related Work

Reinforcement learning has been widely used in learning robot navigation problems. Initial experiments had shown that Q-learning is capable of solving discrete state and action maze navigation problems [4]. Other more recent works have investigated policy gradient algorithms and actor-critic algorithms in continuous control [5]. With the introduction of DQN, high-dimensional problems like playing Atari games using raw pixels as inputs [3] were solvable using deep RL, and later it has been applied to robotic manipulation and navigation [6].

A number of comparative studies have been done to evaluate tabular and function approximation methods in RL. It has been demonstrated that tabular Q-learning is faster to converge in small state spaces but not to large or continuous spaces [7]. DQN and its extensions (Double DQN, Dueling DQN) are more sample efficient and stable in more complicated settings [8], but tend to need careful hyperparameter optimization and high volumes of training data. The present research will add to the literature, as it will directly compare Q-learning and DQN on a gridworld navigation task with fixed obstacles, shedding light on the circumstances, in which each technique is most effective [9][10][11].

Methodology

Environment Configuration

The grid on which the navigation is based is a 10×10 grid. The agent has its initial position (1,1) and a goal (10,10). Ten static obstacle cells are placed in the grid at fixed locations: (3,3), (3,4), (3,5), (4,5), (5,5), (5,6), (6,6), (7,4), (7,5), and (8,5). At each time step, the agent can do four things: move right, left, down, or up. In case an action would cause the agent to be outside of the grid, the agent stays where it is.



The reward system is created to promote effective navigation and punish crashes. The reward values are +100 on reaching the goal, -100 on hitting an obstacle and -1 on a regular time step. The end of each episode is when the agent arrives at the goal or when there are 200 or more steps taken. This reward scheme leads to a sparse reward environment in which the agent has to search to find effective paths.

Q-Learning Implementation

Q-learning is a model-free, off-policy algorithm that learns the optimal action-value function $Q^*(s,a)$ through iterative updates. The Q-table stores a value for each state-action pair and is updated using the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where $\alpha = 0.1$ is the learning rate, $\gamma = 0.9$ is the discount factor, r is the immediate reward, and s' is the next state. The agent selects actions using an epsilon-greedy policy with $\epsilon = 0.1$, meaning it explores randomly 10% of the time and exploits the current Q-values otherwise.

Each time we ran the Q-learning agent, we trained it on 1000 episodes and performed 20 independent runs with various random seeds to evaluate variability. A common obstacle setup and hyperparameters were employed in all the runs. Each run was started with zero in the Q-table.

DQN Implementation

The DQN algorithm approximates the Q-function using a feedforward neural network with two hidden layers of 64 units each and ReLU activation functions. The network takes the agent's (x,y) position as input and outputs Q-values for all four actions. Key components of DQN include:

- **Experience replay:** A replay buffer stores past transitions (s, a, r, s', done) with a capacity of 50,000 samples. During training, mini-batches of 64 transitions are sampled uniformly to break temporal correlations.
- **Target network:** A separate target network with frozen weights is used to compute target Q-values, improving training stability. The target network is updated every 50 episodes by copying the weights from the policy network.
- **Loss function:** The network is trained to minimize the mean squared error between predicted Q-values and target values:

$$L = \mathbb{E}_{(s,a,r,s') \sim \text{buffer}} \left[\left(r + \gamma \max_{a'} Q_{\text{target}}(s', a') - Q(s, a) \right)^2 \right]$$

The DQN agent was trained using the Adam optimizer with a learning rate of 0.001. Training consisted of 500 episodes per run, repeated over 5 independent runs. The exploration rate was fixed at $\epsilon = 0.2$ throughout training. The implementation was developed in Python using PyTorch and a custom gridworld environment.

Evaluation Metrics

We compared the two algorithms in terms of the following metrics:

- **Success rate:** How many episodes out of a maximum number of episodes the agent achieved the goal using the maximum number of steps.
- **Average path length:** Average number of steps undertaken on successful episodes, standard deviation of which is given to show variability.
- **Convergence episode:** The episode where moving average of returns (50 window) was first above a threshold of 50, which showed stability in performance.

These measures offer complementary views on the efficiency of learning, quality and reliability of policies.



Experimental Setup

This section provides information about the hardware and software setup to implement and test the Q-learning and DQN algorithms. All the experiments were performed on a regular desktop computer using Intel Core i7 processor, 16 GB of RAM, and without any specific GPU acceleration. This hardware setup is typical of a typical laboratory environment and shows that the suggested approaches do not need powerful computing equipment.

The implementation of Q-learning was created in MATLAB R2023a relying on the provided numerical and plotting functions. The simulations were all run on Windows 10 and the random seed randomly changed among 20 independent runs to obtain variability in exploration and convergence. The scripts in MATLAB automatically stored learning curves, summary statistics and visualizations of the learned paths as CSV and PNG files to be analyzed further.

The code of the DQN was written in Python based on the PyTorch 2.x deep learning system. The training and evaluation were conducted using a Google colab setup with a CPU backend. NumPy was used in the code to perform numerical operations and Matplotlib was used to plot learning curves. It used a 50,000 transition replay buffer and a mini-batch of 64 to stabilize training, and a target network was updated after every 50 episodes.

The list of the most significant hyperparameters of both algorithms is given in Table X. These values were chosen due to standard practice in the reinforcement learning literature and initial tuning to make sure that learning in the 10x10 gridworld was stable.

Table X. Main hyperparameters for Q-learning and DQN.

- Q-learning:

- Learning rate $\alpha = 0.1$
- Discount factor $\gamma = 0.9$
- Exploration rate $\epsilon = 0.1$ (constant)
- Episodes per run = 1000
- Independent runs = 20
- Maximum steps per episode = 200

- DQN:

- Network architecture: 2 hidden layers, 64 neurons each, ReLU activation
- Learning rate (Adam) = 0.001
- Discount factor $\gamma = 0.9$
- Exploration rate $\epsilon = 0.2$ (fixed)
- Replay buffer size = 50,000 transitions
- Mini-batch size = 64
- Target network update frequency = 50 episodes
- Episodes per run = 500
- Independent runs = 5
- Maximum steps per episode = 200



Results

Q-Learning Performance

The mean success rate of the Q-learning algorithm in 20 independent runs on the 10×10 grid navigation problem with fixed obstacles was 99.11%. This means that the agent could access the goal state in nearly all evaluation episodes when the training converged. In the successful trials, the trained policy took an average of 28.51 ± 19.64 steps to the target which is a relatively short path even though there were obstacles. The standard deviation shows that there is a moderate variability in path length, which is probably because of the stochastic exploration policy and varying paths found during training.

The progress curve of Q-learning depicts that the average return increases rapidly at the initial episodes and after that, the average return stabilizes at a plateau, as the agent has found out the paths close to the goal, which are near optimal. The convergence normally took place in the initial 200 episodes, and thereafter the performance was at a steady level in all the runs. The Q-table representation enabled the agent to store the best actions to take in every state without an error of approximation, which led to the high success rate.

DQN Performance

Conversely, the DQN-based agent that is trained in Python and with a similar environment setup achieved a mean success rate of 52.52% over 5 successive runs with 500 training episodes at a time. The DQN learner only showed slow improvement in episodic returns, but its overall reliability was much lower than that of Q-learning with the same reward structure. The mean convergence episode of the DQN agent was 293.6, which means that it was necessary to have a comparatively long training horizon before the policy could stabilize and generate stable positive returns.

DQN learning curve is more variable and slower improving than Q-learning, and there are numerous deep negative-return spikes due to collisions with obstacles or being stuck in suboptimal loops. These findings indicate that in this low-dimensional and discrete gridworld the neural network approximation added extra complexity that impeded the learning performance. Experience replay mechanism and target network allowed stabilizing training to a certain degree, yet they were not enough to achieve the performance of the tabular approach.

Comparative Analysis

Table 1 summarizes the main performance metrics for both algorithms.

Algorithm	Success rate (%)	Avg. successful steps (mean $\pm$ std)	Mean convergence episode
Q-learning	99.11	28.51 ± 19.64	~150–200
DQN	52.52	N/A	293.6

Q learning and DQN have a significant difference in their performance. Q-learning almost managed to perform the task, giving a very high success rate and fairly short paths. DQN, however, had trouble with consistent navigation and had to go through loads of additional episodes before converging. This distinction can be largely explained by the small state space (100 grid cells) and discrete action space that is more conducive to tabular representations. In this context, neural networks generate generalization but approximation error and slower convergence as a result of functional approximation.

Discussion

The experimental results point to a noticeable dissimilarity in performance between classical Q-learning and the DQN agent in the 10×10 environment, which is considered [12]. Q-learning almost solved the problem, offering a very high success rate and comparatively long paths, which validates that a simple



tabular representation is adequate in case the state space is small and observable in its entirety [13]. The algorithm is able to store and update state-action values directly without approximation error resulting in rapid and constant convergence [14][15].

However, in contrast, the DQN agent did not achieve this performance even when a deeper function approximator was used. The average success rate of 52.52 and the late convergence at episode 293.6 designates that the network needed additional data to effectively estimate the value function and was also delicate to the exploration strategy and hyperparameter choices. Specifically, the negative reward applied to collisions and time steps might have generated a skewed distribution of returns that can slow down training the Q-network. More tuning of network architecture, learning rate, replay buffer size, exploration schedule may be helpful to improve the performance of DQN, but would need a lot of further experimentation.

In general, these findings indicate that deep reinforcement learning may not necessarily be the most effective option to small discrete navigation tasks, where tabular approaches continue to offer high baselines. But the DQN framework is appealing to extend to larger grids, continuous state dynamics, or more complicated vehicle dynamics, where the generalization ability of neural networks can be necessary. Under these conditions, the network architecture, size of the replay buffer, and the exploration schedule will be essential to bring the performance disparity evident in the existing experiments to a minimum.

Conclusion

This study compared tabular Q-learning and Deep Q-Network algorithms for autonomous navigation in a 10×10 gridworld with static obstacles. Q-learning achieved a success rate of 99.11% with an average path length of 28.51 ± 19.64 steps, demonstrating efficient and reliable learning. DQN was less successful (52.52%), and took an average of 293.6 episodes to converge, which is indicative of the complexities of function approximation in small discrete settings.

The findings affirm that tabular Q-learning is a suitable choice when dealing with navigation problems with small state spaces, and DQN might be better when dealing with more extensive or continuous states where tabular algorithms are not practical. The future will seek to study more hybrid methods, bigger grid sizes, dynamic obstacles, and continuous control, to gain a better insight into the trade-offs between classical and deep reinforcement learning in autonomous navigation.

References

- [1] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.
- [2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [3] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.
- [4] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of Artificial Intelligence Research*, vol. 4, pp. 237–285, 1996.
- [5] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. Int. Conf. Machine Learning (ICML)*, 2015, pp. 1889–1897.
- [6] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1–40, 2016.
- [7] G. Tesauro, "Temporal difference learning and TD-Gammon," *Communications of the ACM*, vol. 38, no. 3, pp. 58–68, 1995.



- [8] H. Van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 30, no. 1, pp. 2094–2100, 2016.
- [9] D. Silver *et al.*, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [10] D. Silver *et al.*, “Mastering the game of Go without human knowledge,” *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.
- [11] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2018, pp. 1861–1870.
- [12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [13] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
- [14] R. Munos *et al.*, “Safe and efficient off-policy reinforcement learning,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2016, pp. 1054–1062.
- [15] M. G. Bellemare, W. Dabney, and R. Munos, “A distributional perspective on reinforcement learning,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2017, pp. 449–458.